

Image Processing and Computer Vision PCVK RTI235007

Introduction to Digital Images: Representation , Characteristics ,
and Applications in Computer

TEACHING TEAM
IMAGE PROCESSING AND COMPUTER VISION

Basic Operations of Image Processing

- Point transformations: brightness, contrast, inverse, logarithmic, grayscale
- Arithmetic and logical operations on images
- Gamma correction
- Image depth and PSNR
- Denoising and masking

Linear Brightness

- Operation This do addition pixel value if brightness value is increased , and reduced pixel value if brightness value is lowered.

$$g(x,y) = f(x,y)+b$$

- $g(x,y)$ is pixel value after transformation
- $f(x,y)$ is original
- b is brightness

Linear Brightness



Contrast

- Contrast is level distribution of pixels to road intensity color.

$$g(x, y) = a * f(x, y) + b$$

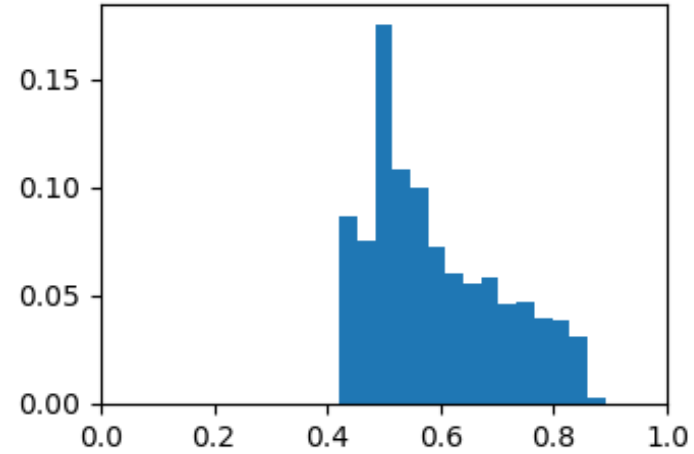
- $g(x, y)$ is pixel value after transformation
- $f(x, y)$ is original
- a is contrast
- b is brightness

Contrast

Low contrast original



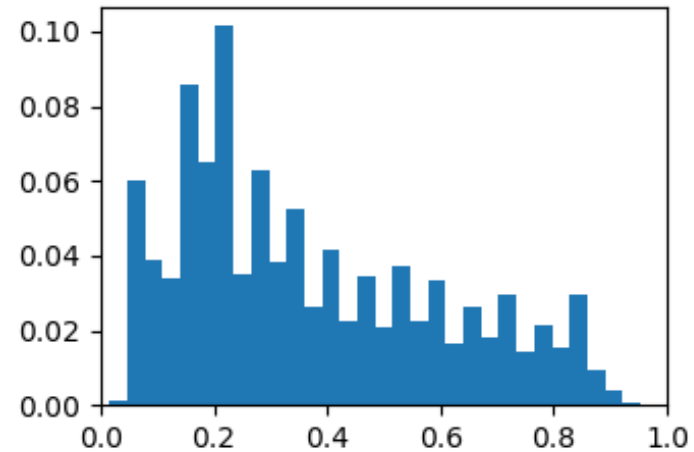
Histogram of low contrast image



Contrast Stretched



Histogram of contrast stretched image



Logarithmic Brightness

- Log Transformation maps something image with color range narrow become more width of the output image .

$$s = c * \text{Log}(1+r)$$

- s : grey-level value of the output
- c : constant
- r : grey-level value of the input

Inverse Color

- The process used For flip mark image with mirror center

$$g(x) = 255 - f(x)$$

- $g(x)$ is negative image
- $f(x)$ is image original

Inverse Color



Grayscale

- Grayscale is image grayish with mark intensity 8-bit pixel

$$Grayscale_{avg} = \frac{(R + G + B)}{3}$$

$$Grayscale_{Lightness} = \frac{\max[R, G, B] + \min[R, G, B]}{2}$$

$$Grayscale_{Luminance} = 0.21R + 0.72G + 0.07B$$

Basic Intensity Transformation Function

- **Inverse Image**

- $S = L-1-r$, with intensity value $[0, L-1]$

- **Log Transformation**

- $S = c \log(1+r)$, c is a constant assuming $r \geq 0$

- **Gamma Transformation (Power-Low)/Gamma Correction**

- $s = cr^\gamma$
- $s = 255 \ c \left(\frac{r}{255} \right)^{1/\gamma}$

Inverse Image

Northpole Image



(a)

Negative of Northpole

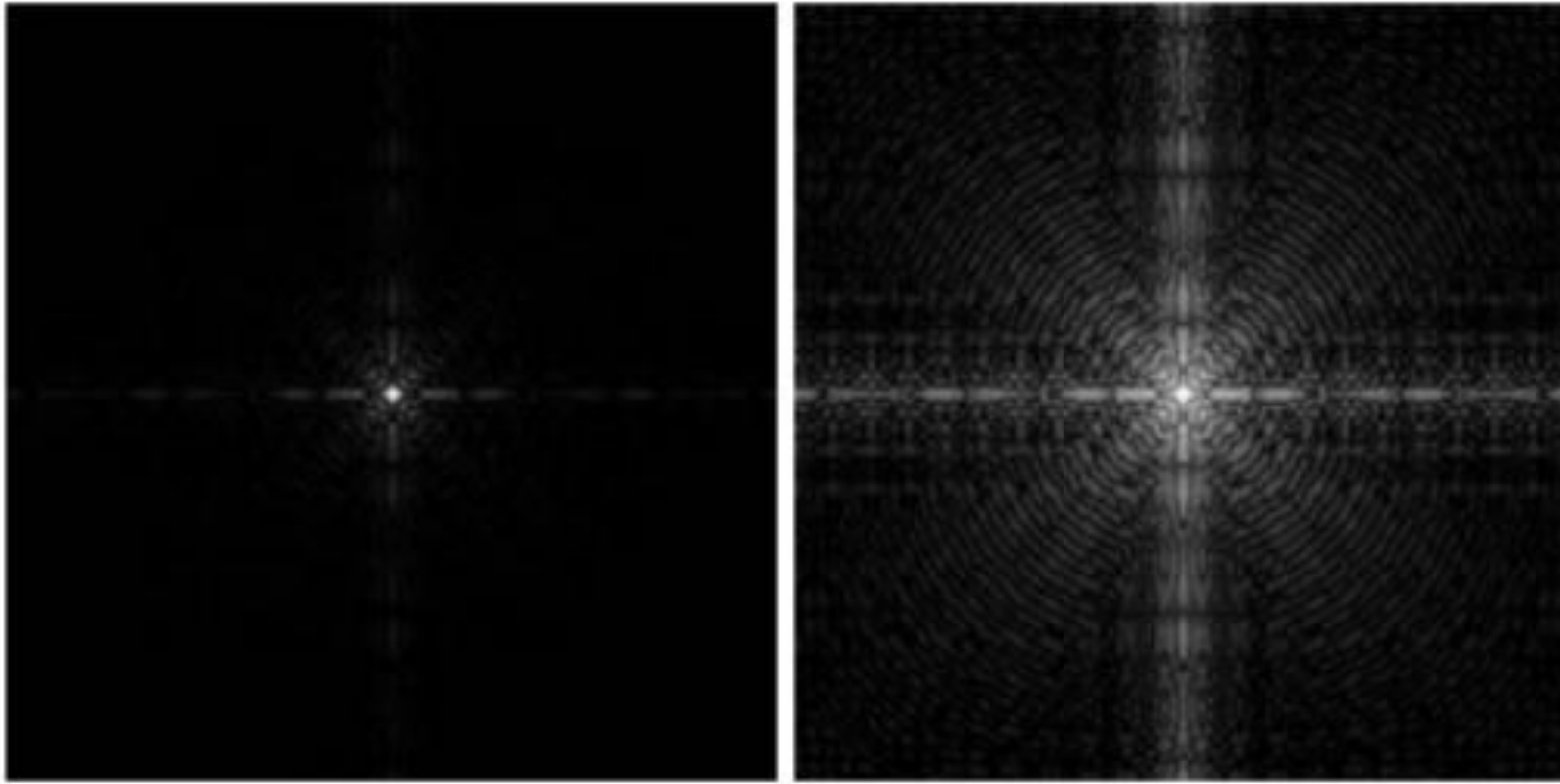


(b)

(a) Northpole image

(b) Negative image of Northpole obtained by Image Inversion

Log Transformation



(a)

(b)

(a) Fourier spectrum

(b) Result of applying the log transformation with $c = 1$

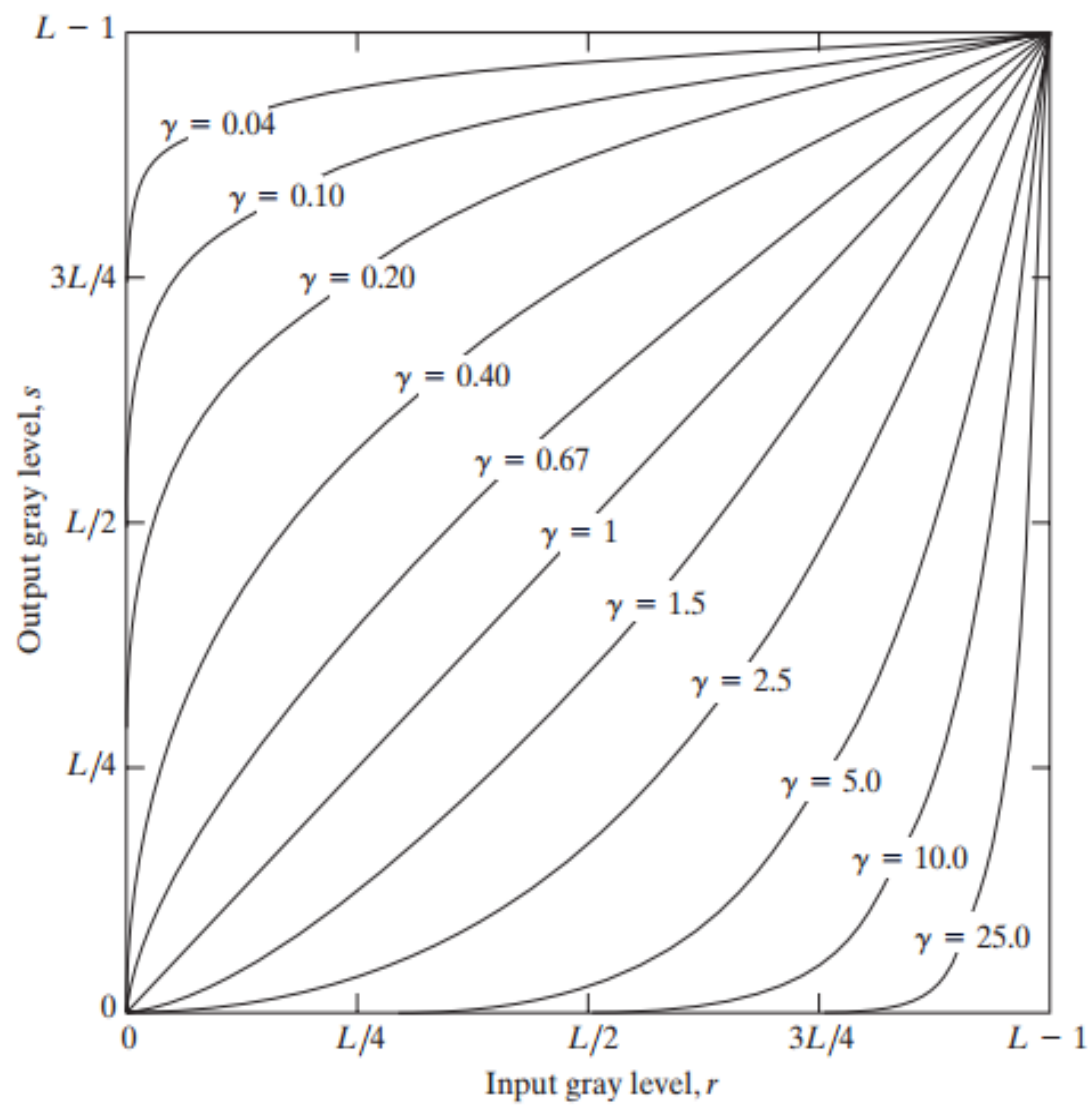
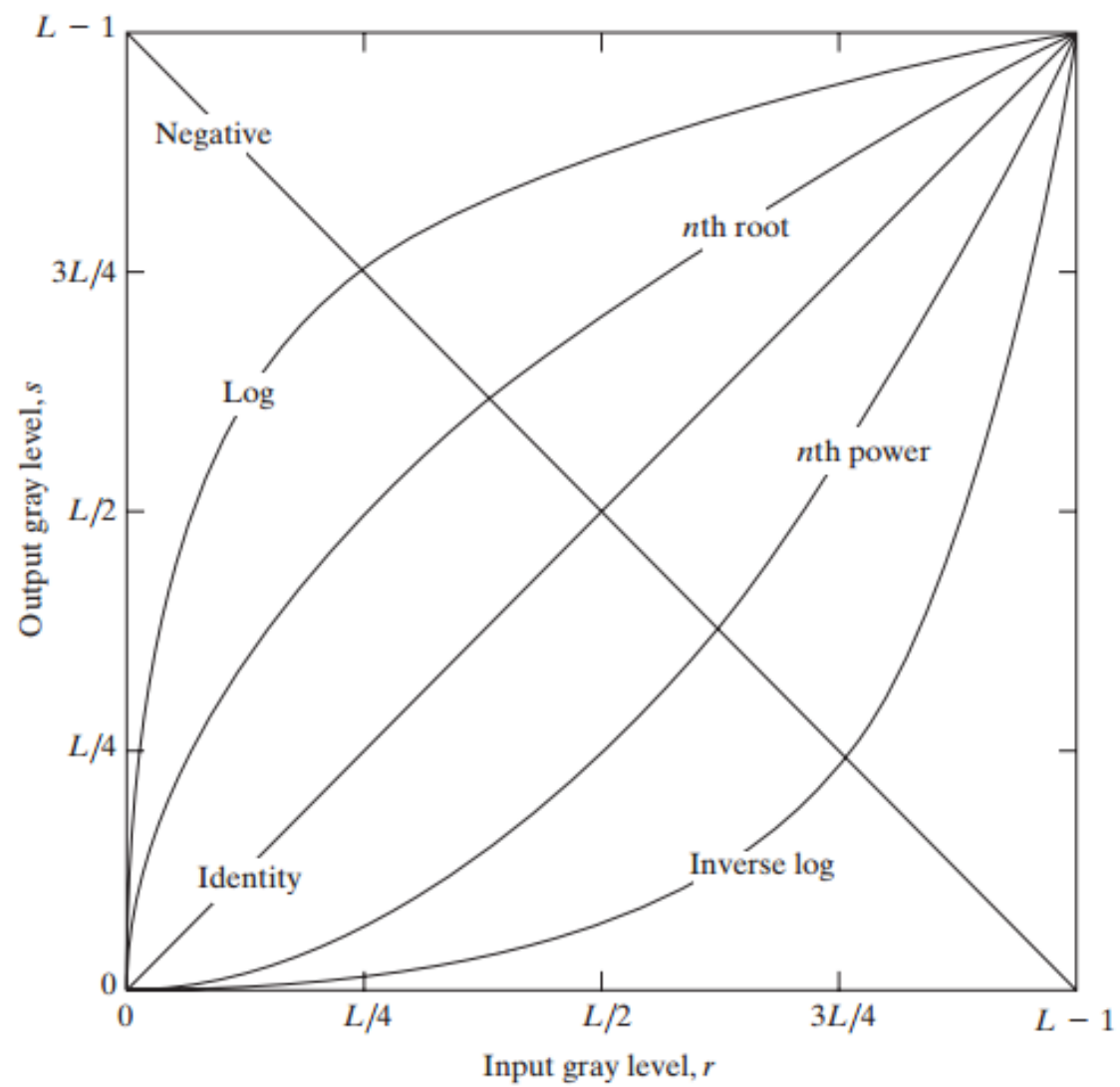
Gamma Correction

- Gamma correction is a technique in digital image operations that is used to improve and correct the brightness and contrast levels in an image.
- This technique is important because monitors and cameras tend not to always produce linear brightness levels.
- Gamma Correction involves using a mathematical function to change the pixel values in an image, compensating for the difference between the actual brightness and the expected brightness in an image.
- Gamma correction is widely used in digital image processing and also in display technologies such as monitors. It is used in a variety of contexts: improving dark images, enhancing image quality, and adjusting the appearance.

Gamma Correction

- Gamma (Power-Law Transformation), symbolized by the Greek letter γ , is described as the relationship between input and the resulting output.
- The input in question is the RGB intensity value of the image. The relationship between the input and output in question is that the output is proportional to the input raised to the power of the gamma value.
- Commonly used gamma values range from 1/2 to 2. Gamma values less than 1 will produce low contrast, while gamma values greater than 1 will produce high contrast.
- The formula for Gamma is as follows:

$$l' = 255x \left(\frac{l}{255} \right)^{1/\gamma}$$



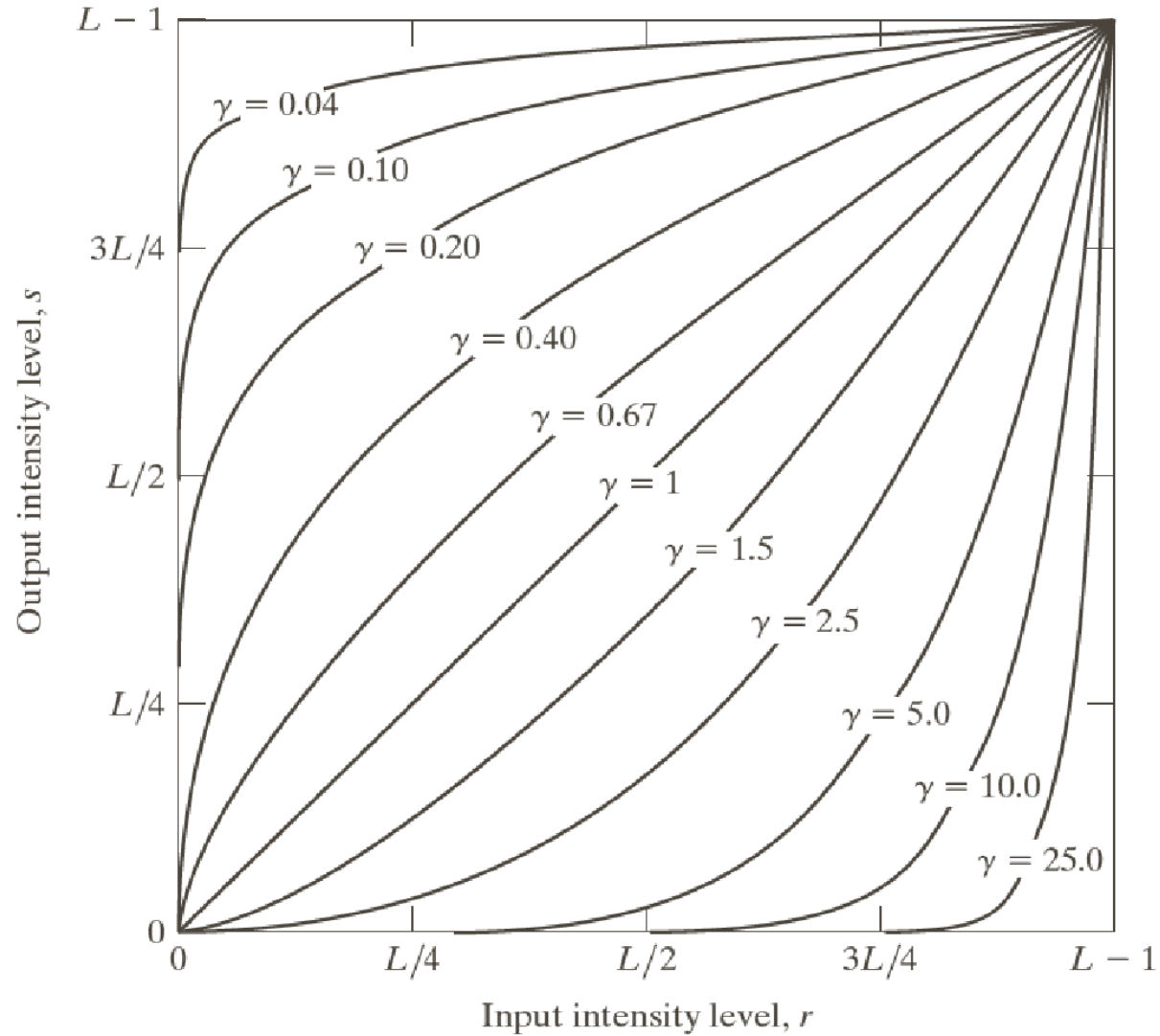
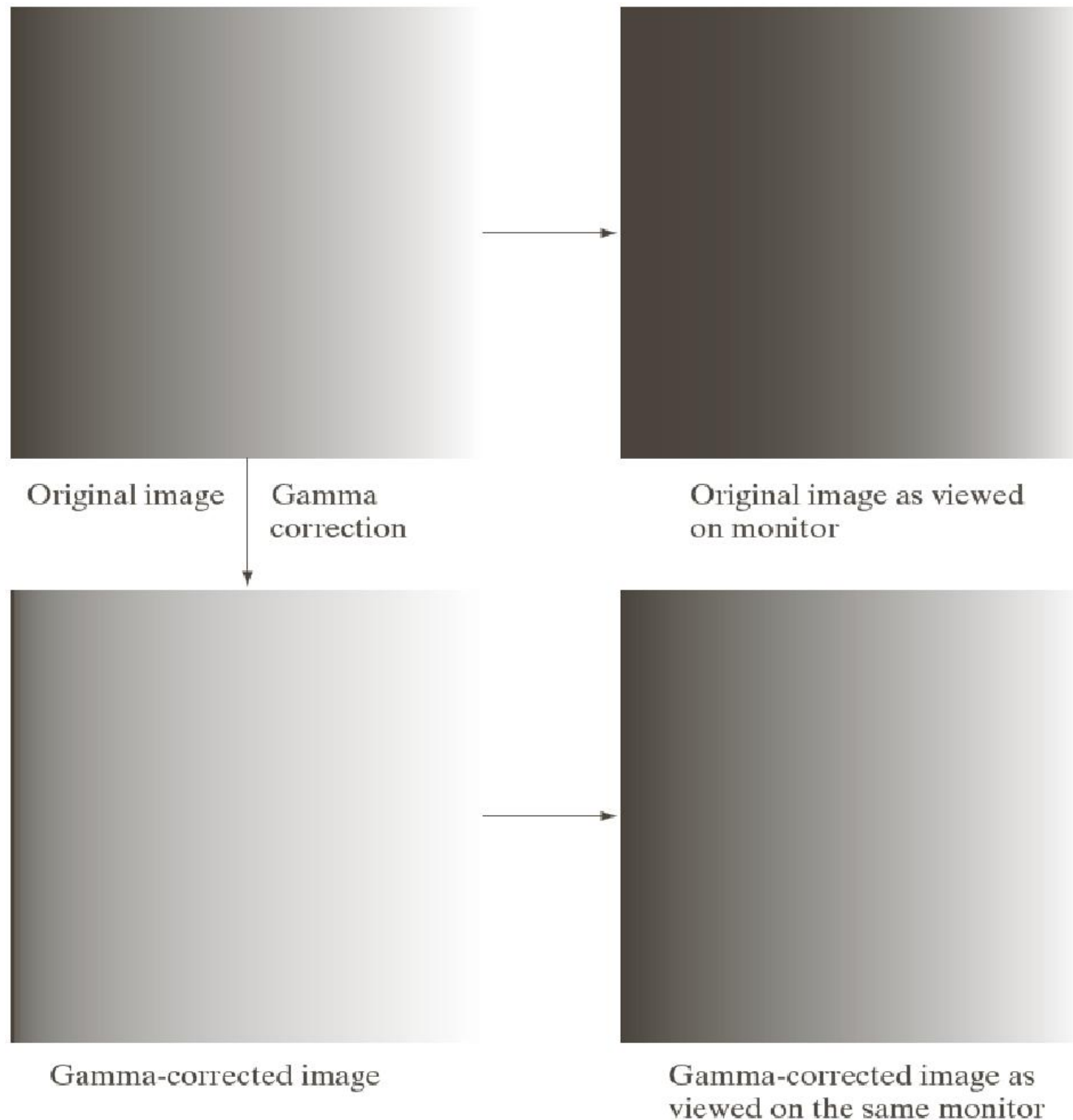
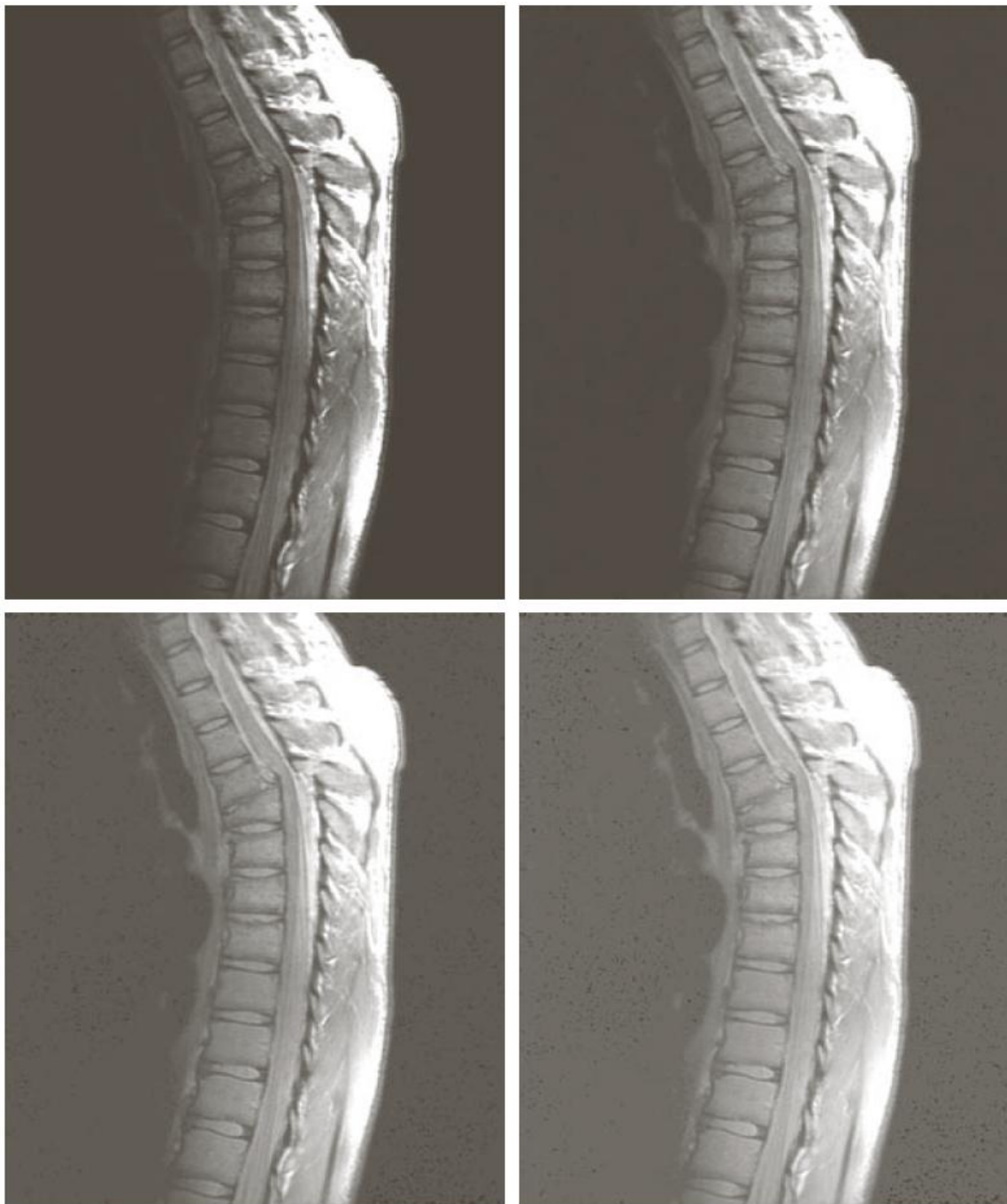


FIGURE 3.6 Plots of the equation $s = cr^\gamma$ for various values of γ ($c = 1$ in all cases). All curves were scaled to fit in the range shown.



a	b
c	d

FIGURE 3.7
(a) Intensity ramp image. (b) Image as viewed on a simulated monitor with a gamma of 2.5. (c) Gamma-corrected image. (d) Corrected image as viewed on the same monitor. Compare (d) and (a).



a	b
c	d

FIGURE 3.8

(a) Magnetic resonance image (MRI) of a fractured human spine.

(b)–(d) Results of applying the transformation in Eq. (3.2-3) with $c = 1$ and $\gamma = 0.6, 0.4$, and 0.3 , respectively. (Original image courtesy of Dr. David R. Pickens, Department of Radiology and Radiological Sciences, Vanderbilt University Medical Center.)



a	b
c	d

FIGURE 3.9

(a) Aerial image.
(b)–(d) Results of applying the transformation in Eq. (3.2-3) with $c = 1$ and $\gamma = 3.0, 4.0$, and 5.0 , respectively. (Original image for this example courtesy of NASA.)

Image Bit-Depth

- Bit-Depth is related to the quantization of the number of bits in an image.
- If using 8-bit quantization, then the number of possible colors is $2^8 = 256$ colors.
- Bit-Depth affects the extent to which an image can reproduce different details and color levels.
- The higher the bit-depth, the more colors can be represented in the image, and the higher the level of detail that can be produced.

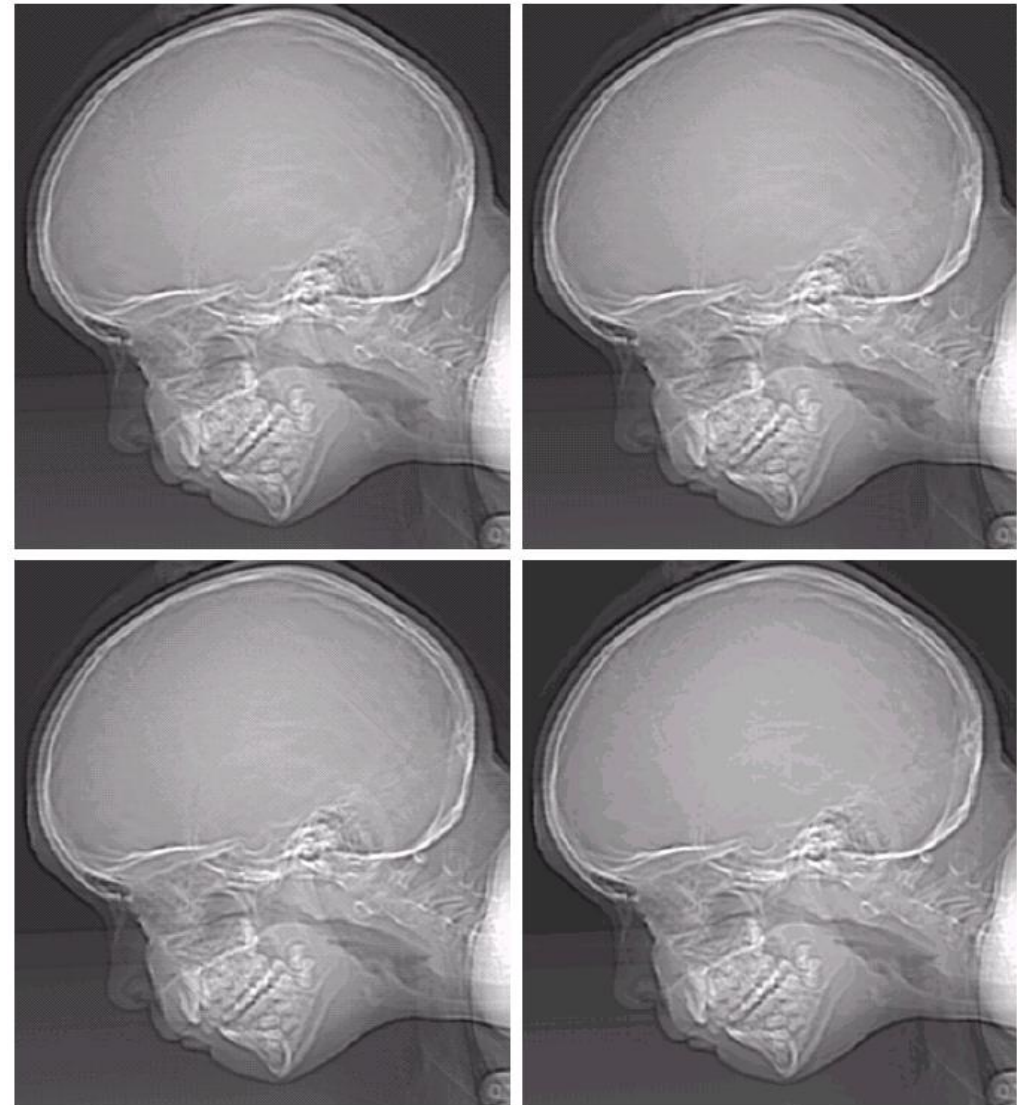
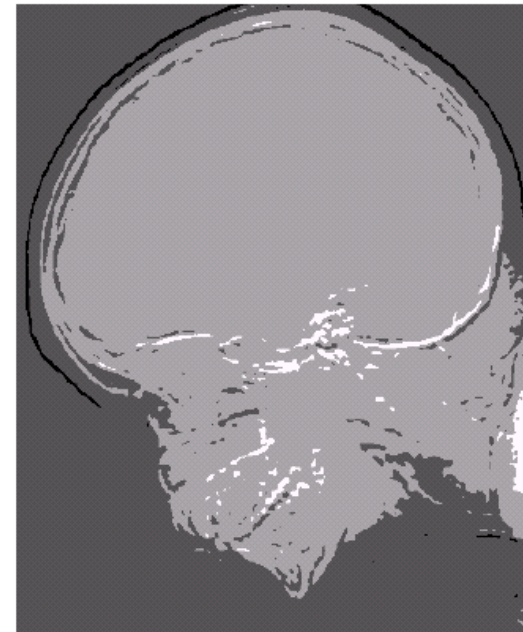
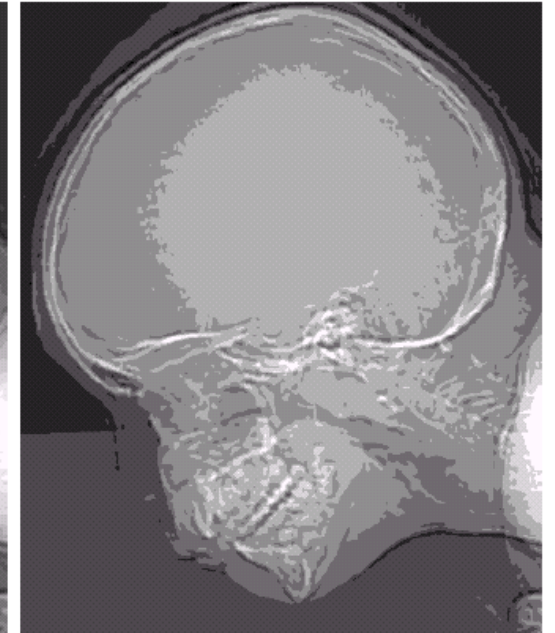


FIGURE 2.21
(a) 452×374 , 256-level image.
(b)–(d) Image displayed in 128, 64, and 32 gray levels, while keeping the spatial resolution constant.

Image Bit-Depth

e f
g h

FIGURE 2.21
(Continued)
(e)–(h) Image displayed in 16, 8, 4, and 2 gray levels. (Original courtesy of Dr. David R. Pickens, Department of Radiology & Radiological Sciences, Vanderbilt University Medical Center.)



Consider the image below. If quantized to 1 bit, then the number of possible colors is only 2: black and white. If quantized to 2 bits, then the number of possible colors is 4 (black, light gray, dark gray, and white).

Image Bit-Depth

- Here is the formula used:

$$level = \frac{255}{2^{bit_depth} - 1}$$

$$C' = round \left(\left(\frac{C}{level} \right) * level \right)$$

Image Bit-Depth

Bit Depth (n)	Varian (biner)	Varian (desimal)	Banyaknya varian (2^n)
1	0,1	0,1	2
2	00,01,10,11	0,1,2,3	4
3	000,001,010,011,100,101,110,111	0,1,2,3,4,5,6,7	8
4	0000,0001,0010,0011,0100...,1111	0,1,2,3,4,5,6,7,...,15	16
5	00000,00001,00010,...,11111	0,1,2,3,4,5,6,7,...,31	32
6	000000,000001,000010,...,111111	0,1,2,3,4,5,6,7,...,63	64
7	0000000,0000001,...,1111111	0,1,2,3,4,5,6,7,...,127	128
8	00000000,00000001,...,11111111	0,1,2,3,4,5,6,7,...,255	256

PSNR

- PSNR (Peak Signal-to-Noise Ratio) is the ratio between the maximum power value of an image and the power of the image affected by noise which affects the quality of the noisy image.
- The higher the PSNR value, the better the image quality.
- To calculate the PSNR of an image, what needs to be done is to compare the denoised/compressed image with the ideal image that has maximum power (in this case the original image).
- PSNR is often used in various applications such as image compression, image processing, and video processing to measure the level of quality degradation that occurs during the compression or processing process.

PSNR

- PSNR is formulated as follows:

$$PSNR = 10 \log_{10} \left(\frac{L^2}{MSE} \right) = 20 \log_{10} \left(\frac{L-1}{RMSE} \right)$$

- L : the number of possible color intensities of the image.
- MSE: mean square error with the formula:

$$MSE = \frac{1}{MN} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} (Original(i, j) - enoised(i, j))^2$$

- RMSE : root mean square error, which is the root of MSE

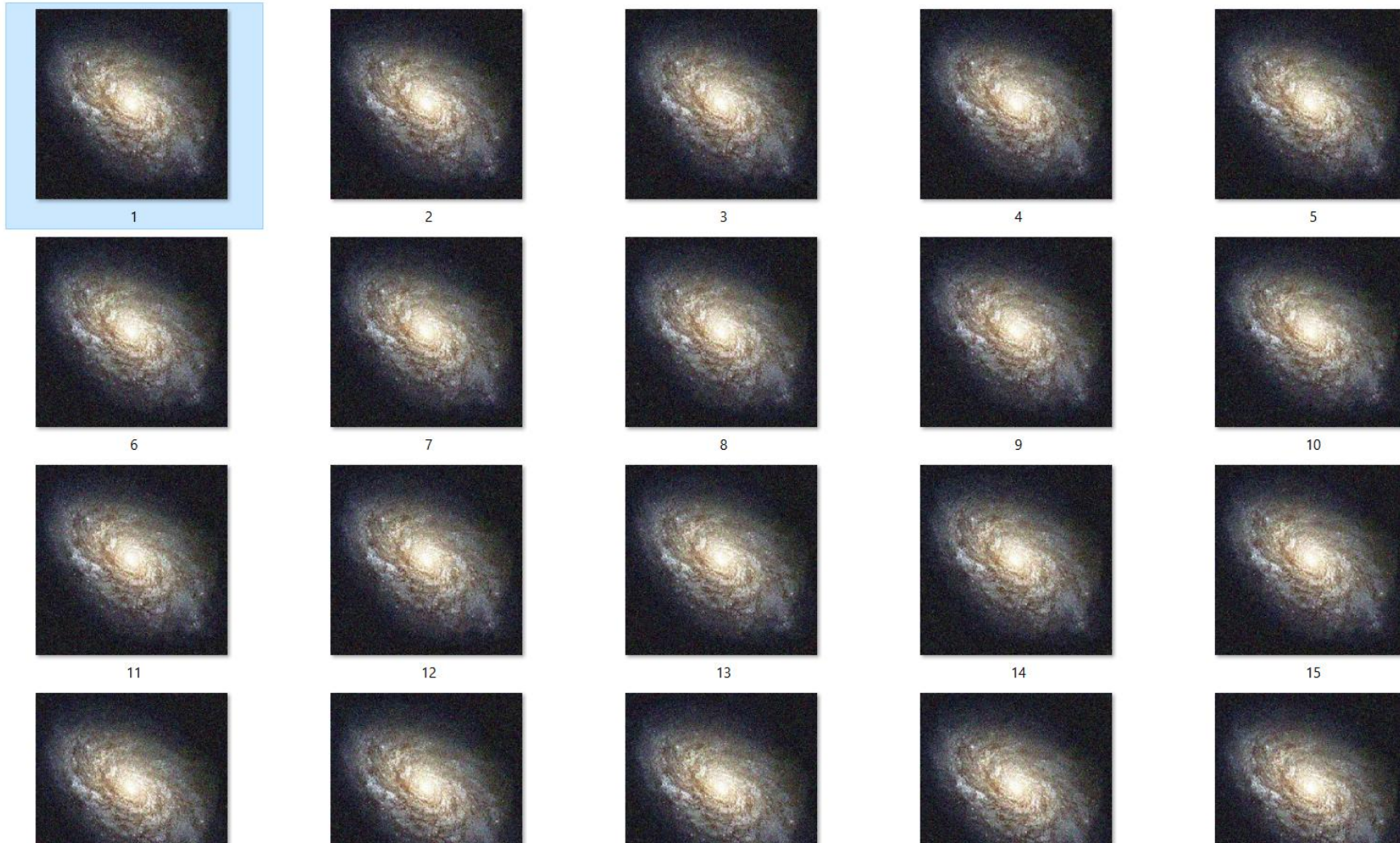
PSNR

```
def PSNR(img1, img2):  
    mse = np.mean((img1 - img2) ** 2)  
    if(mse == 0): # MSE 0 maka tidak ada noise sam  
a sekali, sehingga PSNR tidak memiliki arti  
        return 100  
    max_pixel = 255.0  
    psnr = 20 * log10(max_pixel / sqrt(mse))  
    return psnr
```

Average Denoising

- One of the digital image processing techniques used to reduce noise in images.
- This technique focuses on smoothing an image by replacing a pixel value with the average of a number of surrounding pixel values.
- This operation calculates the average value of each pixel with the same coordinates in a set of images.
- This operation is usually performed to remove noise from external factors that occur during image acquisition.
- Average denoising is usually used on images that have external noise that appears due to the distribution process from source to destination.
- In addition, it is often used in various applications, including medical image processing, image processing, and video processing.

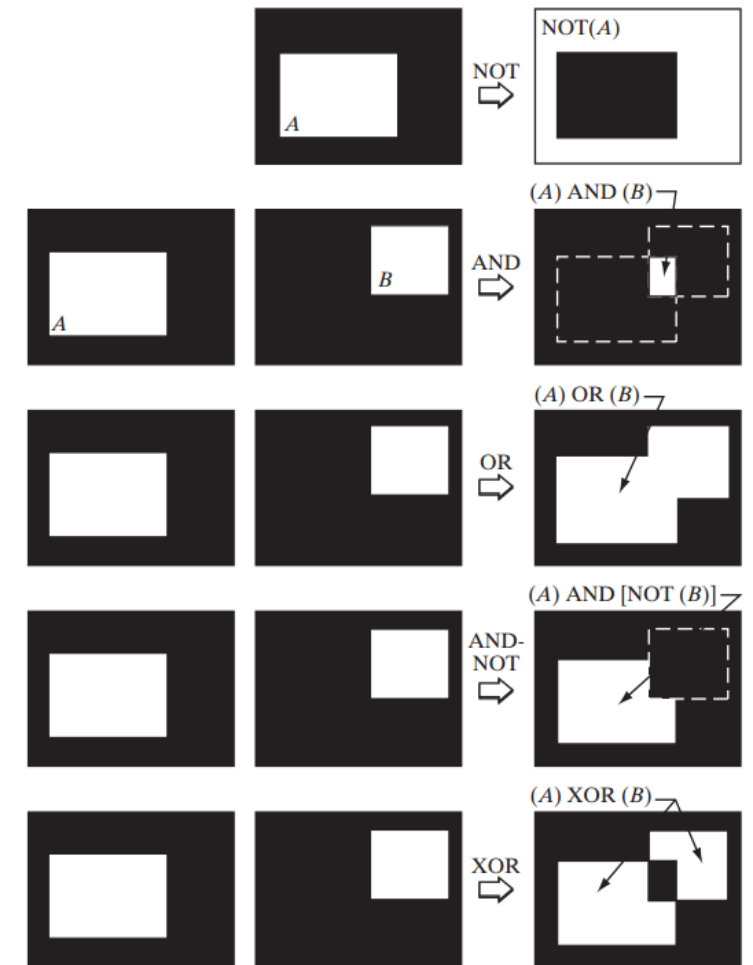
Average Denoising



- The image on the side shows several images containing Gaussian Noise.
- With Average Denoising, a new image will be created that will remove the noise that appears from the noisy images.
- The PSNR value can be used to see the power ratio of the denoised image compared to the original image without noise.

Boolean Operations (Image Masking)

- Image masking is an example of a Logical operation used to process images.
- Some logical operators frequently used in image processing are the OR, AND, and NOT operators. In image masking, the AND operator is the most commonly used operator.
- If it is assumed that two regions (sets) A and B consist of foreground pixels, then the OR operation of these two sets can contain values from A or B. The following figure illustrates Logical operations on images.
- This Boolean operation (Image Masking) is often used in various image processing applications such as object segmentation, medical imaging, edge detection, and many more.



Boolean Operations (Image Masking)

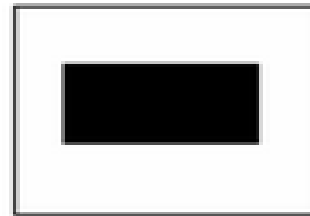
Following is a more detailed explanation of Boolean Operations (Image Masking) in digital image operations:

- **AND (Conjunction):** In the AND operation, a pixel in the resulting image will have a value of 1 (white) only if both the corresponding pixels in the input image and the mask have a value of 1. Used to separate objects from the background.
- **OR (Disjunction):** The OR operation produces a pixel with a value of 1 if one or both pixels in the input image and the mask have a value of 1. Useful for highlighting objects on different backgrounds.
- **NOT (Negation):** The NOT operation produces a result image by inverting the pixel values in the input image. Black pixels become white and vice versa. Used to change the contrast or invert an image.

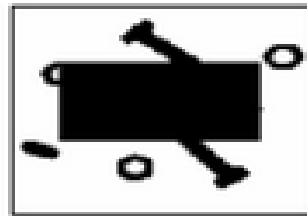
Boolean operations



citra A



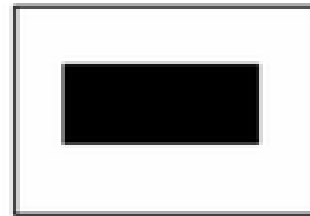
citra B



A AND B



citra A



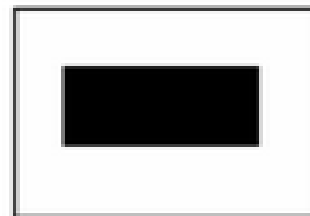
citra B



A OR B



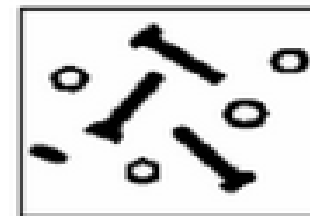
citra A



citra B



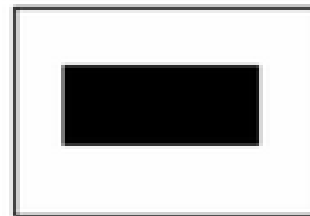
A XOR B



citra A



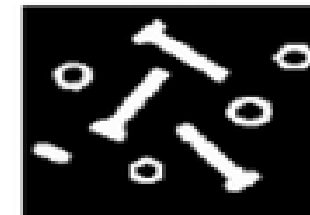
citra A



citra B

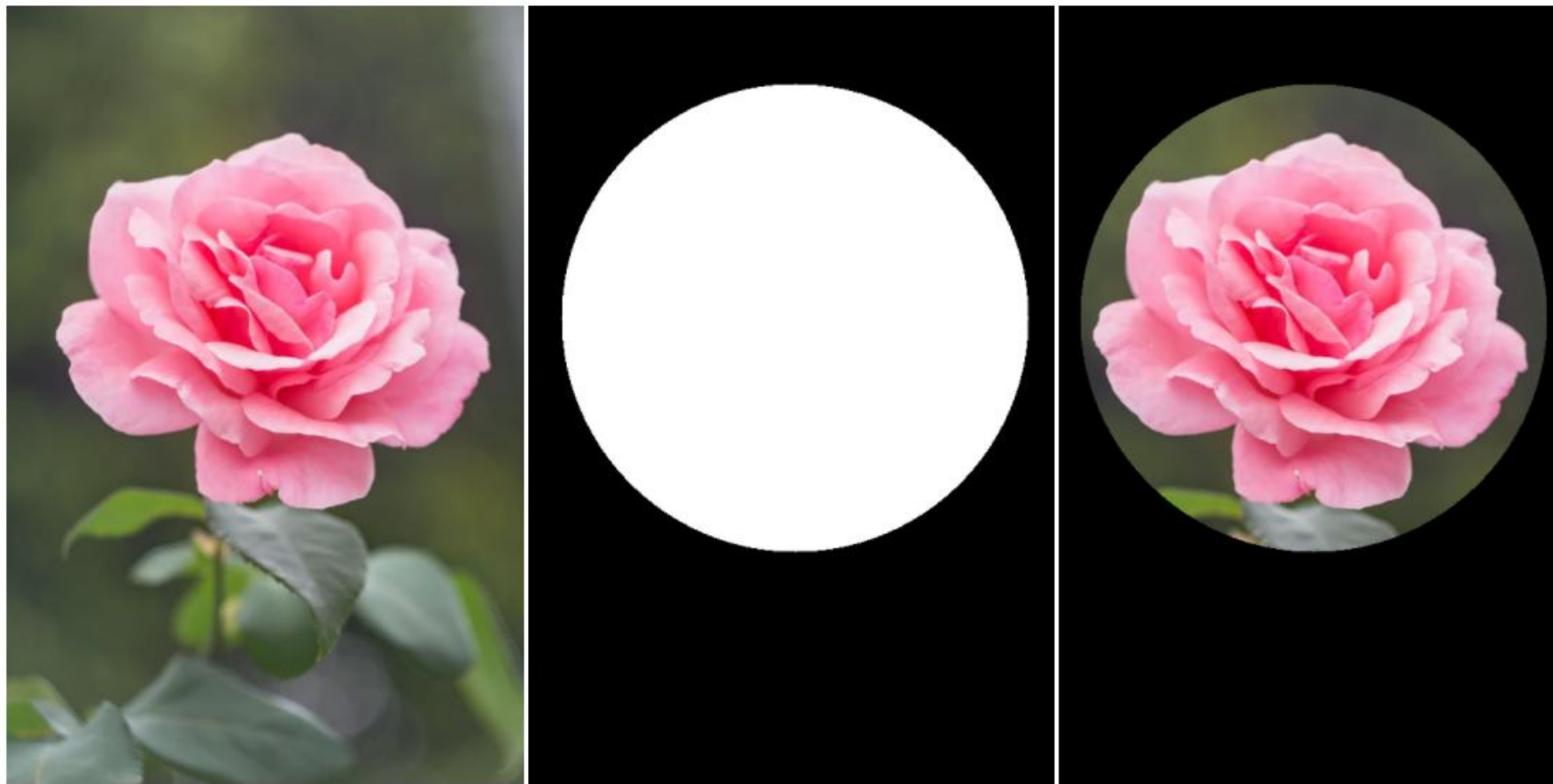


A SUB B



NOT A

AND Image Masking



REFERENCE

Digital Image Processing, Rafael C. Gonzalez and Richard E. Woods

Rayra, H., Yassir, Y., & Rachmawati, R. (2023). USE OF GAMMA CORRECTION WITH ROBERT, PREWITT, AND SOBEL METHODS FOR IMAGE ENHANCEMENT IN UNDERWATER IMAGES. TEKTRON Journal, 7(1), 65-71.

<https://learn.microsoft.com/id-id/windows/win32/direct3ddxgi/using-gamma-correction>

<http://preservationtutorial.library.cornell.edu/tutorial/intro/intro-04.html>

<https://etc.usf.edu/techease/win/images/what-is-bit-depth/>

<https://www.geeksforgeeks.org/denoising-techniques-in-digital-image-processing-using-matlab/>

<https://www.olympus-lifescience.com/en/microscope-resource/primer/java/digitalimaging/processing/imageaveraging/>